Laboratorio 11

Aprender el score y generar imágenes con Langevin

(Denoising score matching)

Valentina Pérez Cadavid

2026

En este caso, se toma una imágen limpia x, se le suma ruido e y se entrena el modelo para que prediga el score $-\frac{e}{\sigma^2}$, la cual es la flecha que devuelve la imágen ruidos a la limpia.

De esta manera se aprende el score sin conocer p_{data}

1. Aplanar imágen original
2. A cada pixel se le suma ruido gaussiano:

$$\vec{y} = \vec{x} + e \quad ; \quad e = \sigma z \quad ; \quad z \sim N(0,1)$$

σ: controla la intensidad del ruido

3. Dada la imágen limpia x, la imágen ruidosa sigue una gaussiana centrada en $\vec{x}$:

$$p(\vec{y}\,|\,\vec{x}) = N(\vec{x}, \sigma^2 I) \qquad \text{Gaussiana en } d = 64 \text{ dimensiones}$$

$$p(\vec{y}\,|\,\vec{x}) = \frac{1}{(2\pi\sigma^2)^{d/2}} e^{-\frac{\|\vec{y}-\vec{x}\|^2}{2\sigma^2}} \quad ; \quad \|\vec{y}-\vec{x}\|^2 = \sum_{i=1}^{d} (y_i - x_i)^2$$

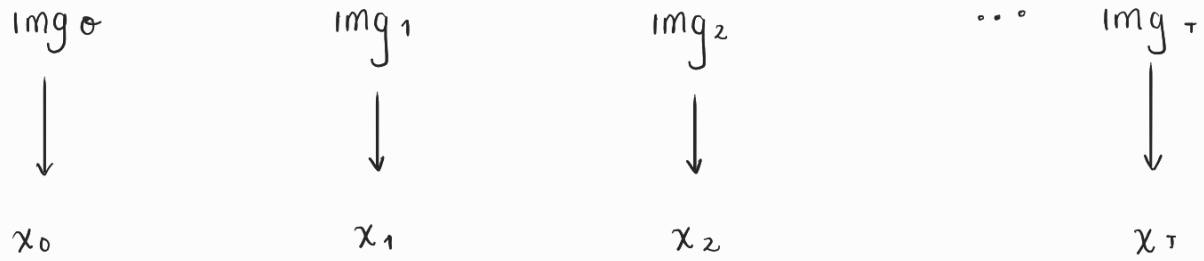
$$\log p(\vec{y}\,|\,\vec{x}) = -\frac{1}{2\sigma^2} \sum (y_i - x_i)^2 - \frac{d}{2} \log(2\pi\sigma^2)$$

$$\nabla_{y_i} \log p(\vec{y}\,|\,\vec{x}) = -\frac{2}{2\sigma^2}(y_i - x_i) = -\frac{1}{\sigma^2}(y_i - x_i) = -\frac{\vec{y}-\vec{x}}{\sigma^2}$$

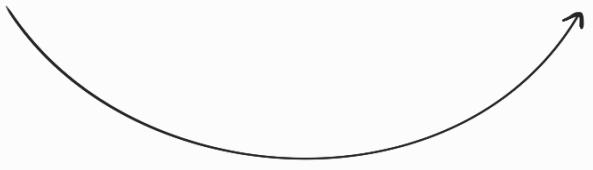
$$\text{Objetivo} \equiv \nabla_{y_i} \log p(\vec{y}\,|\,\vec{x}) = -\frac{\vec{y}-\vec{x}}{\sigma^2} = -\frac{e}{\sigma^2}$$

Corrección correcta

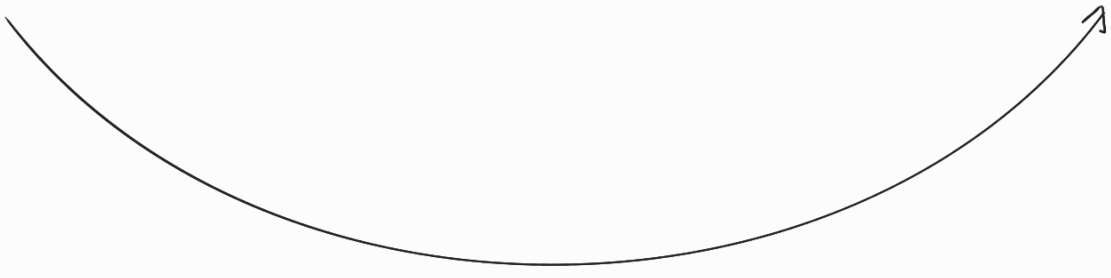
En este caso entonces, entrenamos un modelo para que aprenda a predecir el score (las flechas). Una vez entrenado ese score aprendido, se usa igual al lab 10; es decir, se parte del ruido y con Langevine se llega a una imágen probable



$p(x_1|x_0) = N(x_0, \beta)$ $\quad$ $p(x_2|x_1) = N(x_1, \beta)$



$p(x_2|x_0) = N(x_0, 2\beta)$



$p(x_t|x_0) = N(x_0, t\beta)$

Aprender el score a partir de los datos $\longrightarrow$ Usar el score para generar (langevin)

Lab 11 $\quad$ Lab 10

Langevin = partir de algo al azar y, siguiendo el score más un poco de ruido, llegar a una muestra probable.

Para esto, se usa un modelo simple lineal.

$$S_\theta(y) = A\vec{y} + B \quad ; \quad \theta = (A, B) \leftarrow \text{Parámetros}$$

Se debe minimizar: $L(\theta) = \frac{1}{2}\langle \| S_\theta(\vec{y}) - S_{objetivo} \|^2 \rangle \rightarrow S_{objetivo} = -\frac{e}{\sigma^2}$

$$= -\frac{(y-x)}{\sigma^2}$$

$$= \frac{\vec{x}}{\sigma^2} - \frac{\vec{y}}{\sigma^2}$$

entonces A y B deben ser de la forma: $A = -\frac{1}{\sigma^2}$ $B = \frac{x}{\sigma^2}$

Luego de haber entrenado el modelo, se puede usar para:

1. **Limpiar una imágen ruidosa:** $\vec{x}_{estimada} = \vec{y} + \sigma^2 S_\theta(\vec{y})$; $S_\theta(\vec{y})$: dirección para volver a la img. limpia

   σ: dice cuánto se debe avanzar en esa dirección

2. **Generar una imágen nueva:** $Y_{n+1} = Y_n + \Delta t\, S_\theta(Y_n) + \sqrt{2\Delta t}\, Z_n$

Ejm: Suponer una img. limpia $x = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ $\sigma = 0.5 \rightarrow \sigma^2 = 0.25$

ruido $e = (0.3, -0.2, 0.5, -0.6)$

→ campo objetivo: $S = -\frac{e}{\sigma^2} = -\frac{(0.3, -0.2, 0.5, -0.6)}{0.25} = (-1.2, 0.8, -2.0, 2.4)$

→ imagen perturbada: $y = x + e = (1+0.3, 0-0.2, 0+0.5, 1-0.6) = (1.3, -0.2, 0.5, 0.4)$

→ Comprobar: recuperar la imagen limpia: $x = y + \sigma^2 S$

Pixel	y	S	$\sigma^2 S$	$y + \sigma^2 S$	x original
1	1.3	-1.2	-0.3	1	1 ✓
2	-0.2	0.8	0.2	0	0
3	0.5	-2.0	-0.5	0	0
4	0.4	2.4	0.6	1	1

Con imagenes de perros:

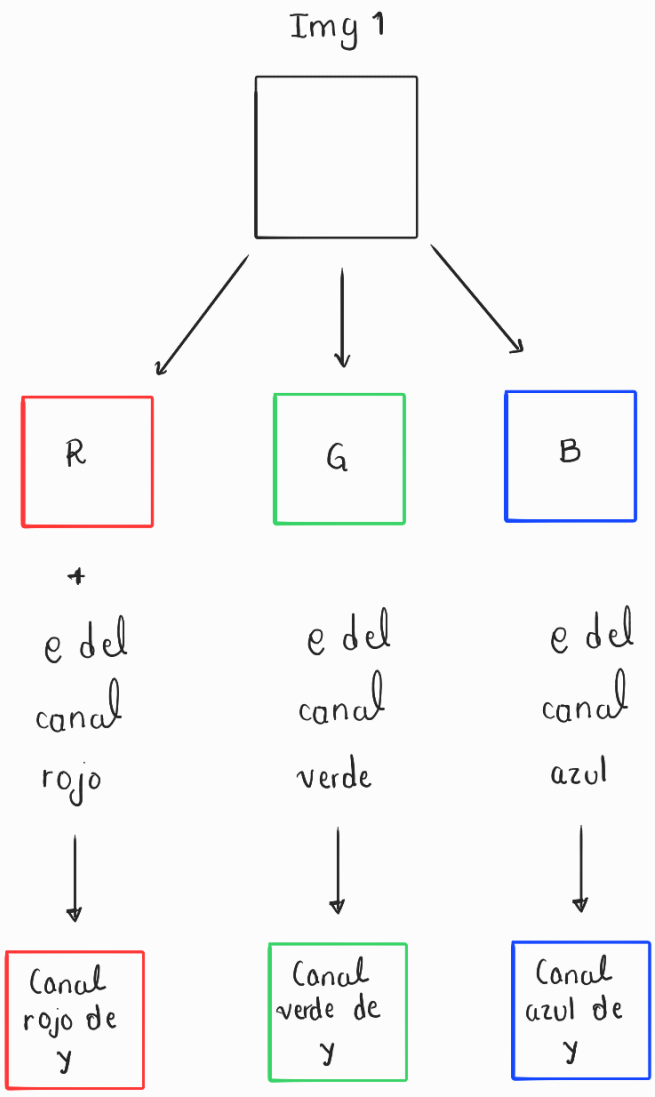
Cada pixel de cada img tiene un ruido diferente

(1000, 32, 32, 3)

- 1000 imágenes
- 32 píxeles de ancho
- 32 píxeles de alto
- 3 canales: RGB

Deben haber 3072000 valores distintos de ruido

$\epsilon \sim N(0, \sigma^2 I)$



Cada canal recibe su propio ruido

Se le muestra al modelo un perro con ruido, y debe decir, pixel por pixel cuánto ruido le asignaron para poder quitarlo.

El modelo solo ve la imágen con ruido Y. No ve el perro limpio ni el ruido. A partir

de Y, debe adivinar: $S_{objetivo} = -\frac{e}{\sigma^2}$

Al saber cual es $S_{objetivo}$, sabrá cuál era el ruido $e = -\sigma^2 S$ y puede recuperar el perro limpio:

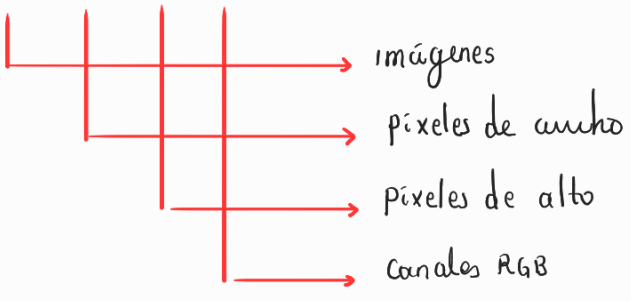
$$x = Y - e = Y + \sigma^2 S$$

Parte 3: Red Neuronal Convolucional

Para mejorar el aprendizaje del modelo, se usa una red neuronal convolucional la cual es capaz de aprender formas y relaciones entre píxeles vecinos.
En este caso se usa Keras para predecir el ruido e, en lugar de predecir el campo

$$S = -\frac{e}{\sigma^2} = -\frac{\sigma Z}{\sigma^2} = -\frac{Z}{\sigma}$$ Usar $\sigma = 0.2$

Para eso se usarán: $(100, 64, 64, 3)$



En este caso, la red aprenderá: Z (ruido estándar sin multiplicar por σ)

$$Z \sim N(0,1), \quad Y = x + \sigma Z.$$

Para esto, la red recibe una flor con ruido Y y devuelve una predicción del ruido Z. Su estructura es:

Entrada → Convolución 1 → Convolución 2 → Salida

- Entrada: Img con ruido — (64, 64, 3)
- Convolución 1: 32 mapas de rasgos simples — (64, 64, 32)
- Convolución 2: 32 mapas de rasgos combinados — (64, 64, 3)
  - Representa los 3 # por cada pixel con ruido
  - Es decir sus canales RGB.
- Salida: Devuelve el ruido que cree que tiene la img.

U - Net

Es una red con forma de U con dos rutas:

Encoder: extrae características de las imágenes

Decoder: reconstruye la resolución original de la imagen

